Atividade 08 — Delivery Tracker: Interface Web com SSR (EJS)

Disciplina: Programação Web II — Backend (Node/Express) · IFAL/Maceió **Capítulo:** 6 — Renderização no Servidor (Server-Side Rendering) com EJS **Modalidade:** Individual · **Peso:** Exercício Cap. 6 (avaliado) **Correção:** automática (autograder SSR por HTTP) + arquitetura/views

> **Pré-requisito:** Exercício Cap. 5 concluído (Delivery Tracker com **Prisma + SQLite**). Você continua **no mesmo repositório** `pweb2-delivery-<matricula>`.

Contexto

Até aqui o Delivery Tracker era uma **API JSON** — ótima para outro programa consumir, mas nenhuma pessoa usa `curl` no dia a dia. Nesta atividade você vai dar uma **cara de site** ao sistema: páginas HTML **renderizadas no servidor** (SSR) com **EJS**, com listagens, formulários e ações por botão.

O ponto central é que a **lógica de negócio não muda**: os `services` e os `repositories` Prisma do Cap. 5 continuam **intactos**. O que muda é a **camada de apresentação** — os `controllers` passam a usar `res.render()` (HTML) em vez de `res.json()`, e surgem as **views**.

> Nesta atividade a interface web **substitui** a API JSON: as rotas passam a servir páginas e formulários. (O endpoint `GET /api/health` continua, apenas como verificação de que o servidor subiu.)

Objetivos de aprendizagem

- Configurar um **view engine** (EJS) com **layout** e **partials** reutilizáveis.
- Construir um **CRUD web**: listagem, formulário, criação e ações de estado.
- Aplicar o padrão **Post-Redirect-Get (PRG)** com **mensagens flash**.
- Validar entrada de formulário e **reexibir o form com erros** (sem perder o que foi digitado).
- Usar **method-override** para disparar **PATCH** a partir de um `<form>` HTML.

O que muda (e o que NÃO muda)

NÃO muda (reaproveite do Cap. 5): - `services/` — **todas** as regras de negócio (transições de status, duplicidade, origem≠destino, atribuição de motorista). - `repositories/*.prisma.js`, `prisma/schema.prisma` e as **migrations**.

Muda: - **Controllers** passam a renderizar views (`res.render(...)`) e a redirecionar após escrever (PRG). - Novas **views EJS** em `src/views/` (layout `base`, partials `navbar` / `flash`, telas de entregas e motoristas, página de erro). - **Rotas** servem páginas e formulários (não mais uma API JSON de recursos). - Middlewares de **validação** (`express-validator`) e de **erros/404** que **renderizam HTML**. - Novas dependências: `ejs`, `express-ejs-layouts`, `express-session`, `connect-flash`, `method-override`, `express-validator`.

Requisitos funcionais — contrato de rotas (SSR)

Implemente **exatamente** estas rotas (o autograder depende delas):

Método (HTTP real)	Caminho	Comportamento esperado
GET	`/`	Redireciona para `/entregas`
GET	`/entregas`	**200** · listagem (tabela) + link **"Nova Entrega"** · filtro opcional por `?status=`
GET	`/entregas/nova`	**200** · formulário com campos `descricao`, `origem`, `destino`
POST	`/entregas`	Cria e **redireciona (3xx, PRG)**. Inválido → **reexibe o form (400)** com as mensagens
GET	`/entregas/:id`	**200** · detalhe com **status** e **histórico**. Inexistente → **404** (página)
PATCH	`/entregas/:id/avancar`	Avança o status e **redireciona (3xx)**
PATCH	`/entregas/:id/cancelar`	Cancela e **redireciona (3xx)**
PATCH	`/entregas/:id/atribuir`	Recebe `motoristaId` (de um `<select>`) e **redireciona (3xx)**
GET	`/motoristas`	**200** · listagem
GET	`/motoristas/novo`	**200** · formulário (`nome`, `cpf`)
POST	`/motoristas`	Cria e **redireciona (3xx)**. CPF duplicado → **reexibe o form** com erro

> As ações de estado (`avancar`, `cancelar`, `atribuir`) são **PATCH**. Como um `<form>` HTML só envia GET/POST, use **method-override**: o form faz `POST .../avancar?_method=PATCH` e a rota é `router.patch(...)`.

Regras obrigatórias: - **RF-01 — View engine:** EJS configurado com **layout** (`layouts/base`) e ao menos os partials `navbar` e `flash`. - **RF-02 — PRG:** toda escrita bem-sucedida responde com **redirect (3xx)**, nunca com HTML direto. - **RF-03 — Flash:** após criar/avançar/cancelar/atribuir, uma **mensagem** aparece na página seguinte. - **RF-04 — Validação:** formulário inválido é **reexibido** com as mensagens e **com os dados digitados** (status **4xx**, nunca 5xx). - **RF-05 — Erros:** entrega inexistente renderiza uma **página 404** (HTML), não um JSON nem um stack trace. - **RF-06 — Regras preservadas:** o comportamento do Cap. 4/5 continua valendo (origem≠destino, duplicada, transições, atribuição só em `CRIADA`, etc.), agora exibido como mensagem na tela.

Convenções de execução (inalteradas)

- `npm start` sobe o servidor, respeita `process.env.PORT`, branch `main`.
- `GET /api/health` → `{ "status": "ok" }` (só readiness).
- `DATABASE_URL="file:./dev.db"` no `.env` (mantenha `.env.example`). `dev.db` e `node_modules/` **fora** do git; **migrations versionadas**.

Como rodar

```
npm install
cp .env.example .env
```

```
npx prisma migrate dev     # banco do Cap. 5
npm run dev                # abra http://localhost:3000 no navegador
```

Correção automática (autograder)

A correção é feita por um autograder que age como um **navegador headless**: confere status HTTP, o **redirect do PRG** e procura trechos no HTML (tabela, formulário, badge de status, histórico).

```
npm start
BASE_URL=http://localhost:3000 node autograder/check.mjs   # meta: 115/115
```

A cada `push`, o **GitHub Actions** executa o autograder (rode `prisma generate` + `prisma migrate deploy` antes de subir o servidor no workflow).

Entregável

- O **mesmo repositório** `pweb2-delivery-<matricula>` (branch `main`), agora com interface web SSR.
- `src/views/` com layout + partials + telas; controllers renderizando HTML.
- `README.md` atualizado (como rodar e um print da tela de entregas).

Critérios (transparentes)

Critério	Peso
Autograder — conformidade das telas e ações (115 pts)	55%
Views: layout + partials + EJS bem usado (escape, includes, loops)	15%
PRG + flash + validação reexibindo o form	15%
Regras de negócio preservadas (reaproveitou services/repos do Cap. 5)	10%
README atualizado (com print)	5%